

web day1

Task 01

```
echo "Hello World!" > readme
↑           ↑           ↑
要輸出的文字 重導向 檔案
```

這句的意思：建立 readme + 把 Hello World! 寫進去。

`echo` 就是「輸出文字」

`cat` 就是把檔案內容顯示出來

`>` 就是覆寫（overwrite）

`>>` 它不是覆寫，而是加到檔案最後面

```
PS C:\Users\weiwei> wsl
weiwei@LAPTOP-ETDQRA93:/mnt/c/Users/weiwei$ cd ~
weiwei@LAPTOP-ETDQRA93:~$ cd Epitech
weiwei@LAPTOP-ETDQRA93:~/Epitech$ mkdir web_day01
weiwei@LAPTOP-ETDQRA93:~/Epitech$ cd web_day01
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task01
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task01
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task01$ echo "Hello World!" > rea
dme
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task01$ echo "Hello"
Hello
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task01$ ls
readme
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task01$ cat readme
Hello World!
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task01$
```

Task 02

`man ->` 就是 manual（使用手冊）。

`man` 就是查看某個指令的說明書

man + 指令 = 查看某個指令的說明。

man man = 查看 man 自己的說明。

```
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task01$ cd ..
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task02
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task02
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task02$ echo "man man" > howto
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task02$ cat howto
man man
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task02$ man
What manual page do you want?
For example, try 'man man'.
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task02$ man man
man: can't resolve man7/groff_man.7
```

```
MAN(1) Manual pager utils MAN(1)

NAME
    man - an interface to the system reference manuals

SYNOPSIS
    man [man options] [[section] page ...] ...
    man -k [apropos options] regexp ...
    man -K [man options] [section] term ...
    man -f [whatis options] page ...
    man -l [man options] file ...
    man -w|-W [man options] page ...

DESCRIPTION
    man is the system's manual pager. Each page argument given to man
    is normally the name of a program, utility or function. The manual
    page associated with each of these arguments is then found and dis-
    played. A section, if provided, will direct man to look only in
    that section of the manual. The default action is to search in all
    of the available sections following a pre-defined order (see DE-
    FAULTS), and to show only the first page found, even if page exists
    in several sections.

    The table below shows the section numbers of the manual followed by
    the types of pages they contain.

    1 Executable programs or shell commands
    2 System calls (functions provided by the kernel)
    3 Library calls (functions within program libraries)
    4 Special files (usually found in /dev)
    5 File formats and conventions, e.g. /etc/passwd
    6 Games
    7 Miscellaneous (including macro packages and conventions), e.g.
      man(7), groff(7), man-pages(7)
    8 System administration commands (usually only for root)
    9 Kernel routines [Non standard]

    A manual page consists of several sections.

    Conventional section names include NAME, SYNOPSIS, CONFIGURATION,
    Manual page man(1) line 1 (press h for help or q to quit)
```

Task 03

1. `ls -a`

`a` = all → 也就是全部都顯示，包括隱藏檔案。

Linux 裡面以 `.` 開頭的通常是隱藏檔案，例如：

```
.git  
.bashrc
```

普通 `ls` 可能看不到，`ls -a` 可以

2. `ls -l`

`l` = long format → 也就是顯示詳細資訊。

會看到類似：

```
-rw-r--r-- 1 weiwei weiwei 1234 Oct 5 10:20 test.txt
```

而不是只有：

```
test.txt
```

3. `ls -h`

`h` = human-readable → 是讓「檔案大小」更容易看懂。

例如原本可能是：

```
1048576
```

搭配 `-h` 後可能顯示成（也就是不用自己算那一大串 bytes）：

```
1.0M
```

→ 三個合再一起 `ls -alh`

例如：

```
-rw-r--r--
```

把它切成：

```
- | rw- | r-- | r--  
↑   ↑   ↑   ↑  
類型 檔案擁有者 群組成員 其他人
```

其中：

```
r = read      = 可以讀  
w = write     = 可以修改  
x = execute   = 可以執行  
- = 沒有這個權限
```

最前面的第一個字則是檔案類型：

```
- = 一般檔案  
d = directory (資料夾)  
l = symbolic link (捷徑)
```

所以例如：

```
drwxr-xr-x
```

先看第一個：

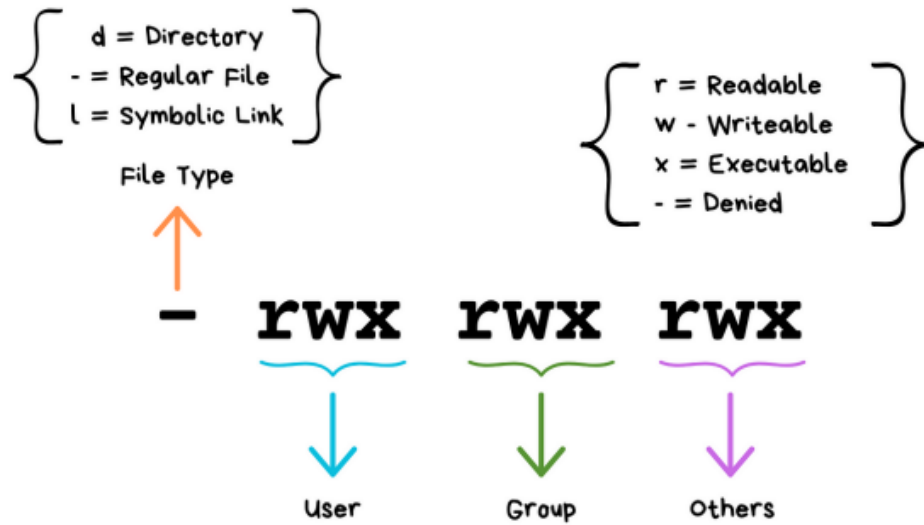
```
d
```

→ 這是一個資料夾。

剩下：

```
rwX | r-X | r-X
```

就是不同人的權限。



```

weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task03
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task03$ echo "ls -alh" > listing
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task03$ cat listing
ls -alh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task03$ ls -alh
total 12K
drwxr-xr-x 2 weiwei weiwei 4.0K Oct  5 10:46 .
drwxr-xr-x 5 weiwei weiwei 4.0K Oct  5 10:45 ..
-rw-r--r-- 1 weiwei weiwei  8 Oct  5 10:46 listing
  
```

Task 04

`.sh` : 裝著一堆 Shell 指令的程式檔

`touch toto.txt` -> 建立 toto.txt

shebang? → `#!/bin/bash`

<code>#!</code>	<code>/bin/bash</code>
↑	↑
shebang	Bash的位置

這個檔案裡面的程式碼，請用 Bash 來執行。

例如：`#!/usr/bin/env python3` → 告訴系統，這個 script 請用 Python 3 執行

加上 execute 權限

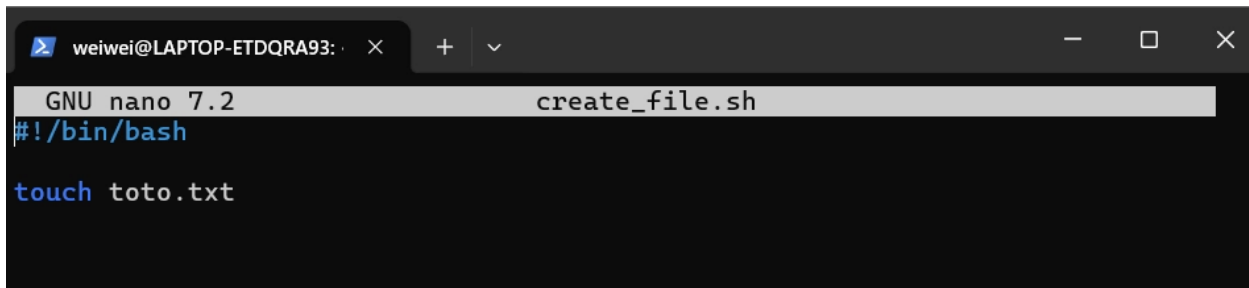
```
chmod  +x  create_file.sh
  ↑    ↑    ↑
改權限 加執行權限 哪個檔案
```

`chmod` = change mode, 用來修改檔案權限。

`+x` :

+ = 加上
x = execute

→ 幫 `create_file.sh` 加上「可以執行」的權限。



```
weiwei@LAPTOP-ETDQRA93: ~$ nano create_file.sh
GNU nano 7.2 create_file.sh
#!/bin/bash

touch toto.txt
```

```
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task04
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task04
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task04$ nano create_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task04$ ls -l
total 4
-rw-r--r-- 1 weiwei weiwei 28 Oct  5 11:01 create_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task04$ chmod +x create_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task04$ ls -l
total 4
-rwxr-xr-x 1 weiwei weiwei 28 Oct  5 11:01 create_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task04$ ./create_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task04$ ls
create_file.sh  toto.txt
```

`./create_file.sh` -> 執行「目前資料夾裡的 `create_file.sh`」

`./` = 目前這個目錄 (current directory)

Task 05

```
weiwei@LAPTOP-ETDQRA93: x + v
GNU nano 7.2 write_in_file.sh
#!/bin/bash

echo "overwrite" > toto.txt
```

```
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task05
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ nano write_in_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ nano write_in_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ echo "Hello World" > toto.txt
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cat toto.txt
Hello World
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ ./write_in_file.sh
-bash: ./write_in_file.sh: Permission denied
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ chmod +x write_in_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ ls -l
total 28
drwxr-xr-x 2 weiwei weiwei 4096 Oct  5 09:50 task01
drwxr-xr-x 2 weiwei weiwei 4096 Oct  5 10:18 task02
drwxr-xr-x 2 weiwei weiwei 4096 Oct  5 10:46 task03
drwxr-xr-x 2 weiwei weiwei 4096 Oct  5 11:03 task04
drwxr-xr-x 2 weiwei weiwei 4096 Oct  5 11:18 task05
-rw-r--r-- 1 weiwei weiwei  12 Oct  5 11:23 toto.txt
-rwxr-xr-x 1 weiwei weiwei  41 Oct  5 11:21 write_in_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mv write_in_file.sh task05/
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ rm toto.txt
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task05
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task05$ ls -l
total 4
-rwxr-xr-x 1 weiwei weiwei 41 Oct  5 11:21 write_in_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task05$ echo "Hello World" > toto
.txt
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task05$ cat toto.txt
Hello World
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task05$ ./write_in_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task05$ cat toto.txt
overwrite
```

Task 06

`mkdir -p music/house music/techno` 在music資料夾裡建立house和techno

<code>mkdir</code>	<code>-p</code>	<code>music/house</code>	<code>music/techno</code>
↑	↑	↑	↑
建立	父目錄	第一個路徑	第二個路徑

要有 `-p` 才可以一次建立多個資料夾的路徑

```
weiwei@LAPTOP-ETDQRA93: ~$ nano create_dirs.sh
GNU nano 7.2 create_dirs.sh
#!/bin/bash

mkdir -p music/house music/techno

weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task06$ nano create_dirs.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task06$ chmod +x create_dirs.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task06$ ./create_dirs.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task06$ ls
create_dirs.sh  music
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task06$ tree
.
├── create_dirs.sh
└── music
    ├── house
    └── techno

4 directories, 1 file
```

Task 07

```
./search_in_file.sh  apple  test.txt
    ↑                ↑      ↑
    程式             $1      $2
```

\$1 = 要搜尋的文字
\$2 = 要搜尋哪個檔案

```
grep "apple" names.txt
```

- 去 `names.txt` 裡找 `apple`
- `grep "$1" "$2"` (去 `$2` 指定的檔案裡，搜尋 `$1` 指定的文字)

```
weiwei@LAPTOP-ETDQRA93:  x  +  v  -  □  ×
GNU nano 7.2 search_in_file.sh
#!/bin/bash

grep "$1" "$2"
```

```
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task06$ cd ..
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task07
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task07
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task07$ nano search_in_file.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task07$ chmod +x search_in_file.s
h
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task07$ nano test.txt
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task07$ ./search_in_file.sh apple
test.txt
apple
pineapple
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task07$ ./search_in_file.sh banan
a test.txt
banana
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task07$ nano search_in_file.sh
```

Task 08

`*.js` 又是什麼？

這個 `*` 代表任何名稱，只要最後是 `.js`

```
find      .      -name      "*.js"
↑         ↑         ↑         ↑
尋找      從目前這個資料夾 名稱是      所有.js
```

`sed` = stream editor 可以用來尋找文字並替換。

例如：

```
sed 's/apple/orange/g' test.txt
```

```
s / apple / orange / g
↑   ↑       ↑       ↑
替換 找這個 換這個 全部
```

s = substitute (替換)

g = global, 代表一行裡如果出現很多次, 全部都換。

但是如果是執行以上的話，它通常只是把修改後的結果輸出到 Terminal。原本的 `test.js` 並沒有真的被修改。

→ 所以需要：

```
sed -i 's/myMoule/myModule/g' test.js
```

i = in-place → 可以理解成直接修改原本的檔案。

兩者結合，找 js 檔然後找要被替換的文字，並替換

→

```
find . -name "*.js" -exec sed -i 's/myMoule/myModule/g' {} \;
```

↑ / ↑ / / / ↑

找出所有.js 對找到的檔案執行後面的指令 -exec要執行的指令到這裡結束

```

GNU nano 7.2 batch_replace.sh
#!/bin/bash

find . -name "*.js" -exec sed -i 's/myMoule/myModule/g' {} \;|

```

```

weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task07$ cd ..
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task08
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task08
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ nano batch_replace.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ chmod +x batch_replace.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ nano test.js
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ nano hello.js
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ cat test.js
const myMoule = "hello";
console.log(myMoule);
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ ./batch_replace.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ cat test.js
const myModule = "hello";
console.log(myModule);
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ nano hello.js
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ nano test.js
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ ^C
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ cd ..
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task08
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ cat hello.js
myModule

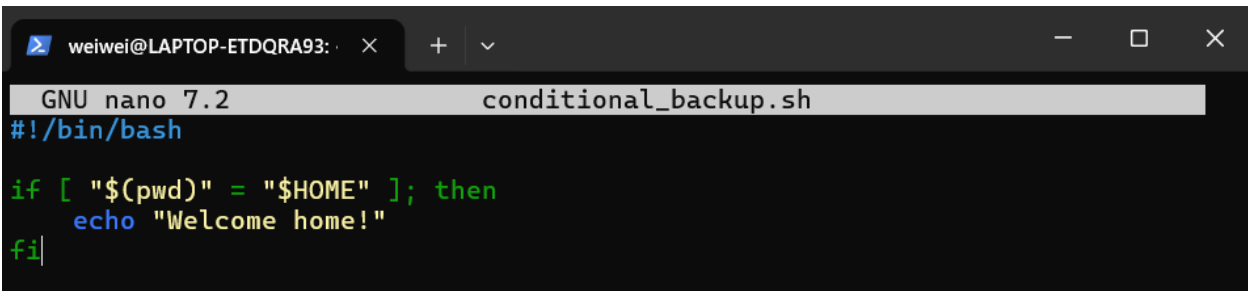
```

Task 09

`$(pwd)` 是什麼？ → 執行 `pwd`，然後拿它的結果來使用

`$HOME` = Home Directory

`fi` = 這個 if 到這裡結束



```

GNU nano 7.2 conditional_backup.sh
#!/bin/bash

if [ "$(pwd)" = "$HOME" ]; then
    echo "Welcome home!"
fi

```

```

weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task08$ cd ..
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task09
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task09
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task09$ nano conditional_backup.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task09$ chmod +x conditional_backup.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task09$ ./conditional_backup.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task09$ cd ~
weiwei@LAPTOP-ETDQRA93:~$ ./Epitech/web_day01/task09/conditional_backup.sh
Welcome home!

```

Task 10

拆分成三部分：

1. `cat log.txt` → 讀取log.txt
2. `grep -i "error"` → 找error，不分大小寫
`i` = ignore case → 忽略大小寫
3. `head -n 5` → 顯示資料的前五筆

head	-n	5
↑	↑	↑
取前面	行數	5

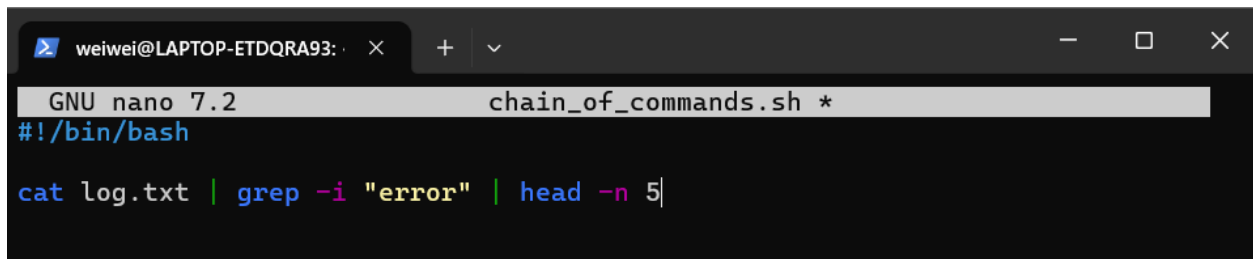
| (pipe) = 把左邊的輸出 → 傳給右邊當輸入

> → 輸出丟進「檔案」

| → 輸出丟給「下一個指令」

三個合在一起：

```
cat log.txt | grep -i "error" | head -n 5
```



```
weiwei@LAPTOP-ETDQRA93: × + v
GNU nano 7.2 chain_of_commands.sh *
#!/bin/bash
cat log.txt | grep -i "error" | head -n 5|
```

```
weiwei@LAPTOP-ETDQRA93:  × + ▾
GNU nano 7.2 log.txt
System started
ERROR: Database failed
User logged in
Error: Network failed
Warning: Low memory
error: File missing
ERROR: Connection lost
System running
Error: Permission denied
ERROR: Timeout
ERROR: Another error|
```

```
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task10
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task10
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task10$ nano chain_of_commands.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task10$ chmod +x chain_of_commands.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task10$ nano log.txt
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task10$ ./chain_of_commands.sh
ERROR: Database failed
Error: Network failed
error: File missing
ERROR: Connection lost
Error: Permission denied
```

Task 11

for → 開始迴圈
do → 開始執行這些事情
done → 迴圈結束

```
weiwei@LAPTOP-ETDQRA93:  × + ▾
GNU nano 7.2 loop_files.sh *
#!/bin/bash

for file in *.md
do
    echo "This is a new line." >> "$file"
done|
```

```
#!/bin/bash
```

```
for file in *.md
do
    echo "This is a new line." >> "$file"
done
```

\$file → 要取得裡面的值

例如：`echo "$file"`

就會輸出：`apple.md`

\$1 → 取得第一個參數的值
\$file → 取得 file 變數的值

```
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task11
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task11
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task11$ nano loop_files.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task11$ chmod +x loop_files.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task11$ echo "Apple" > test1.md
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task11$ echo "Banana" > test2.md
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task11$ cat test1.md
Apple
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task11$ ./loop_files.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task11$ cat test1.md
Apple
This is a new line.
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task11$ cat test2.md
Banana
This is a new line.
```

Task 12

step1:

```
#!/bin/bash

task_count=$1
day=$2

day_number=$(printf "%02d" "$day")
```

```
mkdir "day$day_number"
```

```
printf "%02d" "$day"
```

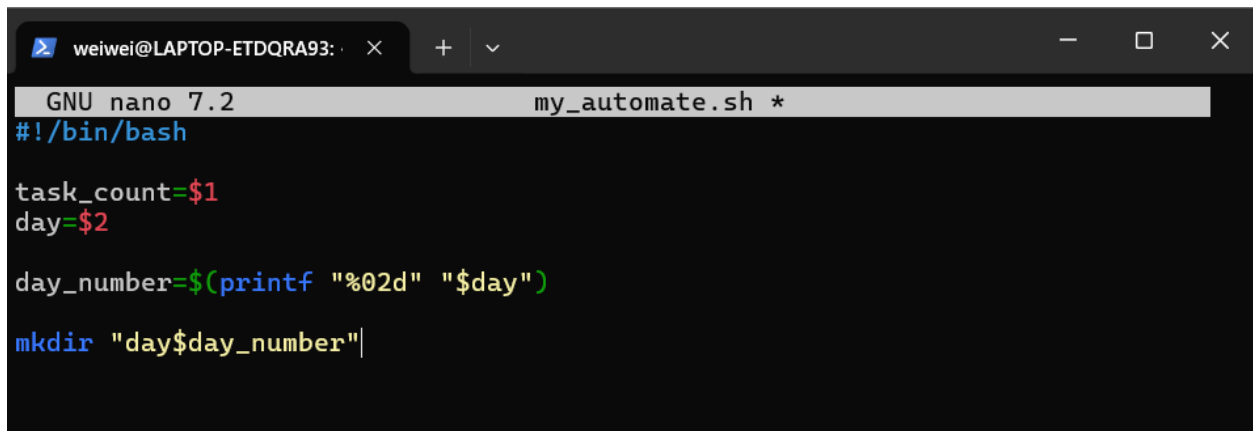
```
|||
```

```
||└ d = decimal integer (整數)
```

```
|└ 2 = 總共至少兩位
```

```
└ 0 = 不足的位置補 0
```

所以 `printf "%02d" 8` → 08



```
weiwei@LAPTOP-ETDQRA93: ~$ nano my_automate.sh
GNU nano 7.2 my_automate.sh *
#!/bin/bash

task_count=$1
day=$2

day_number=$(printf "%02d" "$day")

mkdir "day$day_number"
```

step2:

```
#!/bin/bash
```

```
task_count=$1
```

```
day=$2
```

```
day_number=$(printf "%02d" "$day")
```

```
mkdir "day$day_number"
```

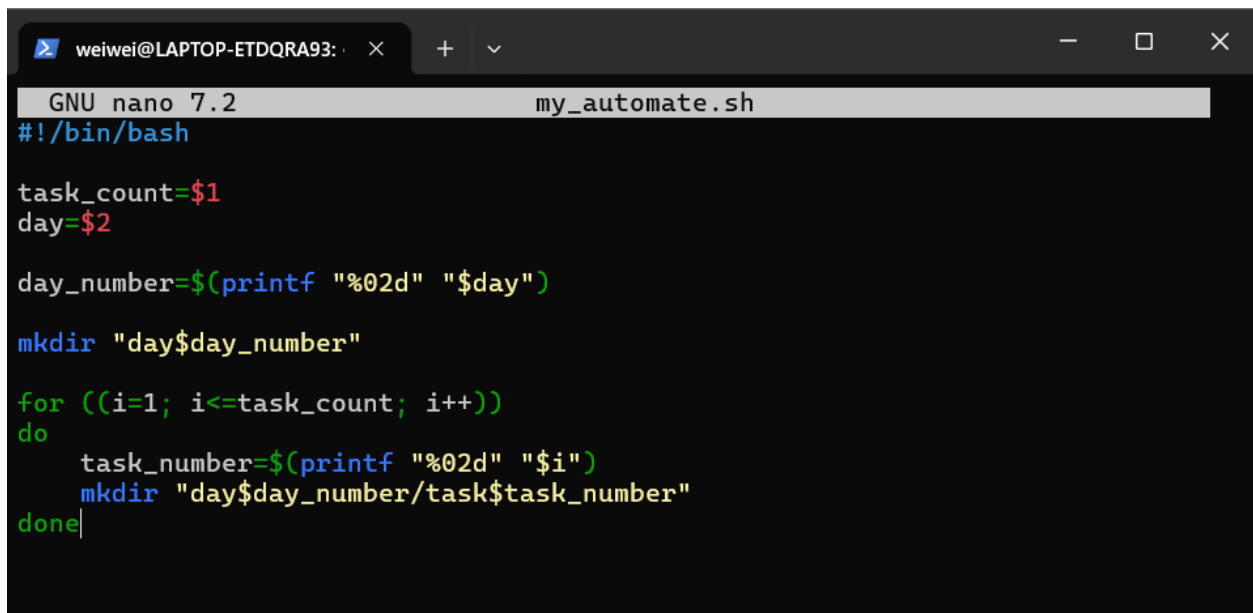
```
for ((i=1; i<=task_count; i++))
```

```
do
```

```
    task_number=$(printf "%02d" "$i")
```

```
mkdir "day$day_number/task$task_number"  
done
```

```
i=1 ; i<=task_count ; i++  
  ↑           ↑           ↑  
從1開始    只要i≤12    每次i加1
```



```
GNU nano 7.2 my_automate.sh  
#!/bin/bash  
  
task_count=$1  
day=$2  
  
day_number=$(printf "%02d" "$day")  
  
mkdir "day$day_number"  
  
for ((i=1; i<=task_count; i++))  
do  
    task_number=$(printf "%02d" "$i")  
    mkdir "day$day_number/task$task_number"  
done
```

```
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ mkdir task12
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01$ cd task12
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task12$ nano my_automate.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task12$ chmod +x my_automate.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task12$ ./my_automate.sh 12 3
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task12$ ls
day03  my_automate.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task12$ nano my_automate.sh
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task12$ rm -r day03
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task12$ ./my_automate.sh 12 3
weiwei@LAPTOP-ETDQRA93:~/Epitech/web_day01/task12$ tree day03
day03
├── task01
├── task02
├── task03
├── task04
├── task05
├── task06
├── task07
├── task08
├── task09
├── task10
├── task11
└── task12

13 directories, 0 files
```